Project - Low Level Design

on

Real Estate Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vidhi Yadav	EN22CS3011077
2.	Suhani Pathak	EN22CS301990
3.	Surbhi Jadam	EN22CS3011002
4.	Vanshika Soni	EN22CS3011061
5.	Vinay	EN22CS3011082

Group Name: Group07D9

Project Number: AAI-07

Industry Mentor Name:

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2026

Table of Contents

S.No.	Content
1.	Introduction
	1.1 Scope of the Document
	1.2 Intended Audience
	1.3 System Overview
2.	System Design
	2.1 Application Design
	2.2 Process Flow
	2.3 Information Flow
	2.4 Components Design
	2.5 Key Design Considerations
	2.6 API Catalogue
3.	LLD Demonstration Requirements
	3.1 UML Diagrams
	3.2 LLM Selection & Rationale
	3.3 Approach Toward Problem Statement
4.	References

1. Introduction

This Low-Level Design (LLD) document provides a comprehensive technical blueprint of the Real Estate Support Triage Agent — a production-grade AI-powered system designed to automate the initial response workflow for incoming real estate communications. The document captures the architectural decisions, module-level specifications, data flows, API contracts, and UML representations of the system built using Python 3.14, LangChain, and Google Gemini 2.5 Flash.

1.1 Scope of the Document

This document covers the following aspects of the system:

- Functional decomposition of all agent modules (Classifier, NER Extractor, Responder)
- System and information flow across the triage pipeline
- REST API specification including endpoint signatures, request/response schemas, and error handling
- Rationale behind selection of Google Gemini 2.5 Flash as the underlying LLM
- UML diagrams including Class Diagram, Sequence Diagram, Use Case Diagram, and Component Diagram
- Approach taken toward solving the problem statement
- Known issues encountered during development and the resolutions applied

1.2 Intended Audience

Audience	Purpose
Software Developers	Understanding module-level design and implementing extensions or integrations
AI/ML Engineers	Reviewing agent prompt engineering, LLM selection, and pipeline architecture
System Architects	Evaluating design decisions, scalability considerations, and information flow
Project Reviewers / Evaluators	Assessing the LLD quality, technical depth, and alignment to the problem statement
QA Engineers	Understanding component boundaries to design effective unit and integration tests

1.3 System Overview

The Real Estate Support Triage Agent is a three-stage reactive AI pipeline that processes any inbound tenant or client message end-to-end without human intervention at the classification or drafting stage. It is designed to solve a high-volume operational bottleneck in property management: the manual reading, sorting, and initial response drafting for every tenant inquiry.

The pipeline consists of three sequential agent tasks:

1. Message Classification — determines the urgency level (HIGH / MEDIUM / LOW) and the intent category (maintenance, inquiry, complaint, payment, viewing)
2. Named Entity Recognition (NER) — extracts structured fields such as tenant ID, unit number, lease date, property address, person name, and issue type
3. Response Generation — produces a professional, empathetic, urgency-appropriate draft reply ready for human review and dispatch

The system is exposed via a FastAPI REST interface, making it consumable by any front-end application, CRM system, or property management platform. All three pipeline stages are powered exclusively by Google Gemini 2.5 Flash via LangChain.

2. System Design

2.1 Application Design

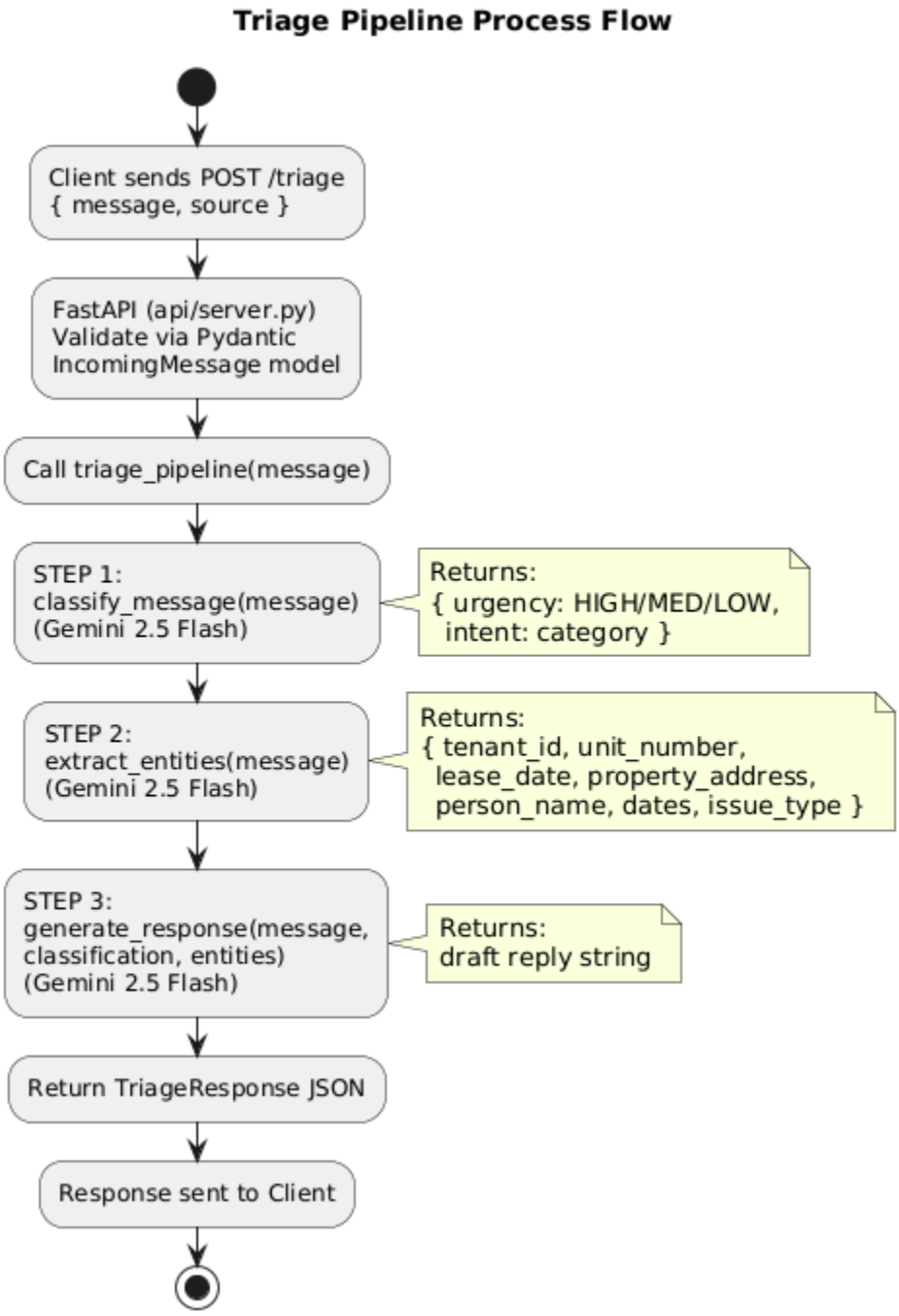
The application follows a modular, single-responsibility architecture. Each agent module is a self-contained Python file exposing exactly one public function. The orchestration layer (main.py) wires these functions in sequence. The API layer (api/server.py) wraps the orchestrator behind a FastAPI endpoint, enabling HTTP access.

Module responsibilities are summarised below:

File	Public Function	Responsibility
agents/classifier.py	*classify_message()*	Accepts a raw message string. Invokes Gemini with a structured system prompt. Returns a JSON dict with urgency and intent fields.
agents/ner_extractor.py	*extract_entities()*	Accepts a raw message string. Invokes Gemini to extract seven named entity fields. Returns a JSON dict with null for absent fields.

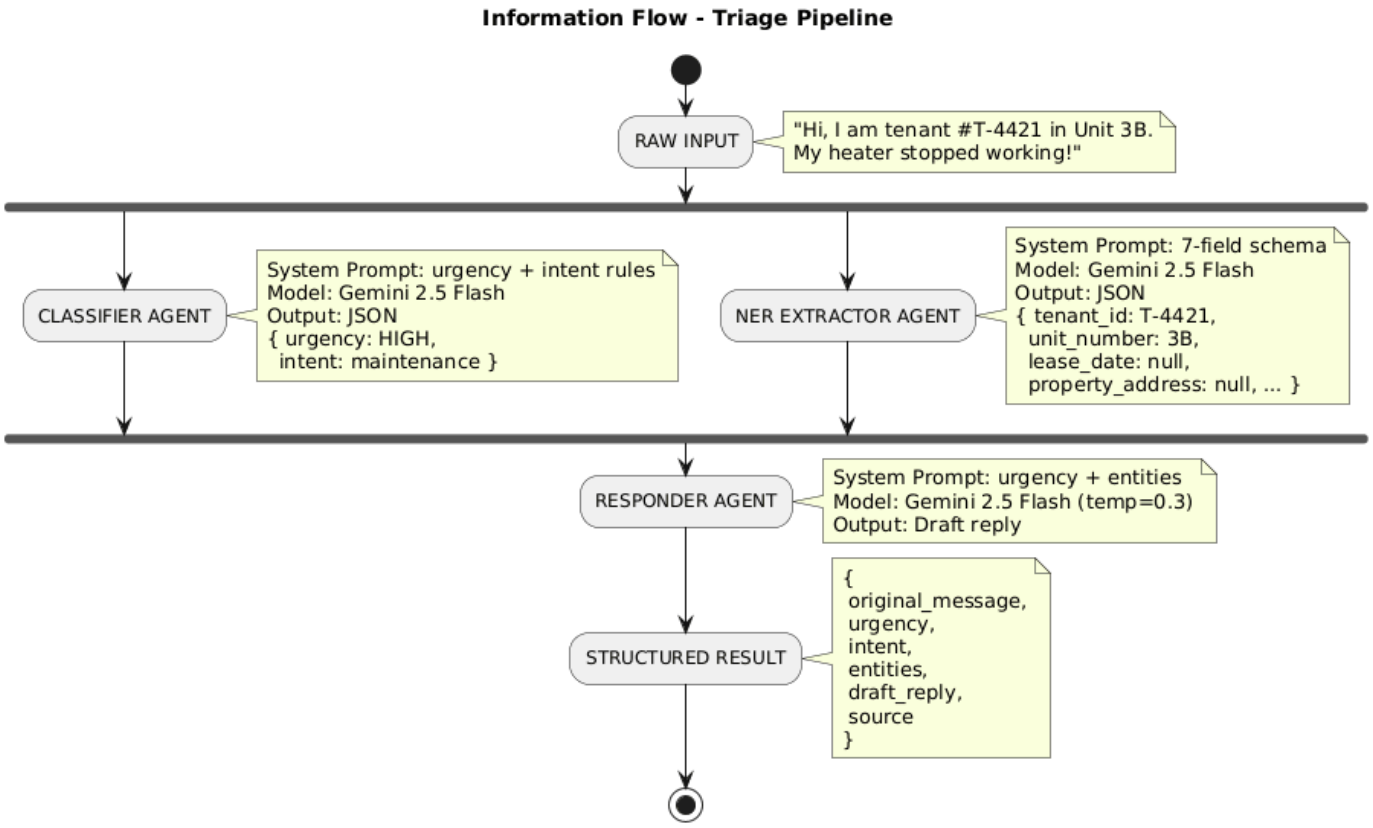
agents/responder.py	*generate_response()*	Accepts message, classification dict, and entities dict. Generates a professional draft reply tailored to urgency and context.
main.py	*triage_pipeline()*	Orchestrates the three agents in sequence. Consolidates outputs into a single result dict. Also serves as a CLI entry point.
api/server.py	*POST /triage*	FastAPI endpoint. Validates input with Pydantic, calls triage_pipeline(), and returns a typed TriageResponse model.
database/models.py	*ORM Models (Base, TriageTicket)*	SQLAlchemy ORM schema for persisting triage tickets. Not yet wired into the pipeline (Phase 8 — pending).
tests/test_agents.py	*Pytest Test Cases*	pytest unit tests for all agent modules covering urgency edge cases, entity extraction, and response format validation.

2.2 Process Flow



2.3 Information Flow

The information flow diagram below shows how data transforms at each stage, from raw unstructured text to a fully structured triage result:



2.4 Components Design

This section describes the internal design of each component.

2.4.1 Classifier Agent (agents/classifier.py)

The classifier is the first stage in the pipeline. It maps an unstructured natural language message to a structured urgency/intent pair.

Key design decisions:

- Temperature set to 0 to ensure deterministic, consistent classification outputs
- System prompt constrains output to exactly five intent categories and three urgency levels
- Urgency classification follows real-world property management triage rules: life-safety and infrastructure failures are always HIGH
- The clean_json() helper sanitises Gemini responses that occasionally wrap JSON in markdown code fences

Input/Output contract:

Direction	Field	Description
Input	*message: str*	Any natural language string representing an incoming tenant or client message
Output	*urgency: str*	One of: HIGH, MEDIUM, LOW
Output	*intent: str*	One of: maintenance, inquiry, complaint, payment, viewing

2.4.2 NER Extractor Agent (agents/ner_extractor.py)

The NER extractor is the second stage. It converts the same raw message into a set of structured entity fields using Gemini as a zero-shot NER engine.

Note: spaCy was originally considered for NER but was removed due to Windows Application Control policy blocking its compiled DLL files. Gemini 2.5 Flash was selected as the sole NER engine.

Output Field	Type	Description
tenant_id	*str* | *null*	Alphanumeric identifier for the tenant (e.g., T-4421)
unit_number	*str* | *null*	Apartment or unit number mentioned in the message (e.g., 3B)
lease_date	*str* | *null*	Any date related to lease commencement or expiry
property_address	*str* | *null*	Full or partial property address if mentioned
person_name	*str* | *null*	Name of the sender or referenced person
dates	*str* | *null*	Any other date references in the message
issue_type	*str* | *null*	Free-text description of the reported issue

2.4.3 Responder Agent (agents/responder.py)

The responder is the third stage. It consumes all outputs from the previous two stages and produces a human-quality draft reply.

Temperature is set to 0.3 (slightly above zero) to allow natural, non-robotic language while keeping the reply professional and on-topic. The system prompt injects the urgency level and entity data as contextual variables, enabling personalised responses. Response time commitments are hardcoded in the prompt by urgency tier:

Urgency	Response Time Committed	Example Trigger
HIGH	Action within 2 hours	Flooding, fire, no heat, lockout
MEDIUM	Action within 24 hours	Broken appliance, noise complaint, billing query
LOW	Action within 3 business days	General question, scheduling, feedback

2.4.4 Orchestrator (main.py)

The triage_pipeline() function is the central coordinator. It imports all three agent functions and calls them in sequence, collecting their outputs into a single consolidated dict. It also serves as the CLI entry point, running three test messages with rate-limit-aware sleep intervals when executed directly.

The function signature and return structure are:

```
def triage_pipeline(incoming_message: str) -> dict:
    classification = classify_message(incoming_message)
    entities       = extract_entities(incoming_message)
    draft_reply    = generate_response(incoming_message, classification, entities)
    return {
        'original_message': incoming_message,
        'urgency':          classification['urgency'],
        'intent':           classification['intent'],
        'entities':         entities,
        'draft_reply':      draft_reply
    }
```

2.4.5 API Server (api/server.py)

The API server wraps the orchestrator behind a FastAPI application. It uses Pydantic models for automatic request validation and OpenAPI documentation generation.

Two Pydantic models govern the API contract:

- IncomingMessage: message (str, required), source (str, default='web')
- TriageResponse: original_message, urgency, intent, entities, draft_reply, source

CORS is configured with allow_origins=['*'] to enable integration with any client. The Swagger UI at /docs provides an interactive browser-based interface for testing the POST /triage endpoint without any additional client tooling.

2.5 Key Design Considerations

Consideration	Design Decision and Rationale
LLM Temperature	Classifier and NER use temperature=0 for deterministic, repeatable outputs. Responder uses temperature=0.3 for natural but controlled language generation.
JSON Sanitisation	The clean_json() helper strips markdown fences that Gemini occasionally wraps around JSON, preventing json.loads() failures in production.
Rate Limiting	Gemini 2.5 Flash free tier limits to 5 RPM. Each triage call makes 3 LLM requests. A time.sleep(30) guard between CLI test messages prevents 429 errors.
Module Isolation	Each agent module imports its own LLM instance and loads .env independently. This makes modules testable in isolation without pipeline context.
Pydantic Validation	FastAPI's integration with Pydantic ensures all API inputs and outputs are type-validated, preventing malformed requests from reaching the pipeline.
Database	SQLAlchemy + SQLite schema is defined in database/models.py but not yet wired. This deliberate deferral avoids blocking the core pipeline on infrastructure decisions.
Secret Management	API keys are stored in a .env file and loaded via python-dotenv. The .env file must be added to .gitignore to prevent accidental credential exposure in version control.

2.6 API Catalogue

All endpoints are served by the FastAPI application running at http://127.0.0.1:8000. Interactive documentation is auto-generated at /docs via Swagger UI.

Method	Endpoint	Auth	Description
GET	*/*	None	Returns running status and agent name. Health probe for load balancers.
GET	*/health*	None	Returns { status: 'healthy' }. Dedicated health check endpoint.

POST	*/triage*	None (API key planned for Phase 10)	Main triage endpoint. Accepts a JSON body with message and source fields. Returns full triage result.
GET	*/docs*	None	Auto-generated Swagger UI. Allows interactive browser-based testing of all endpoints.

POST /triage — Request Body (IncomingMessage):

```
{
 "message": "Hi, I am tenant #T-4421 in Unit 3B. My heater stopped working!",
 "source":  "web"
}
```

POST /triage — Response Body (TriageResponse):

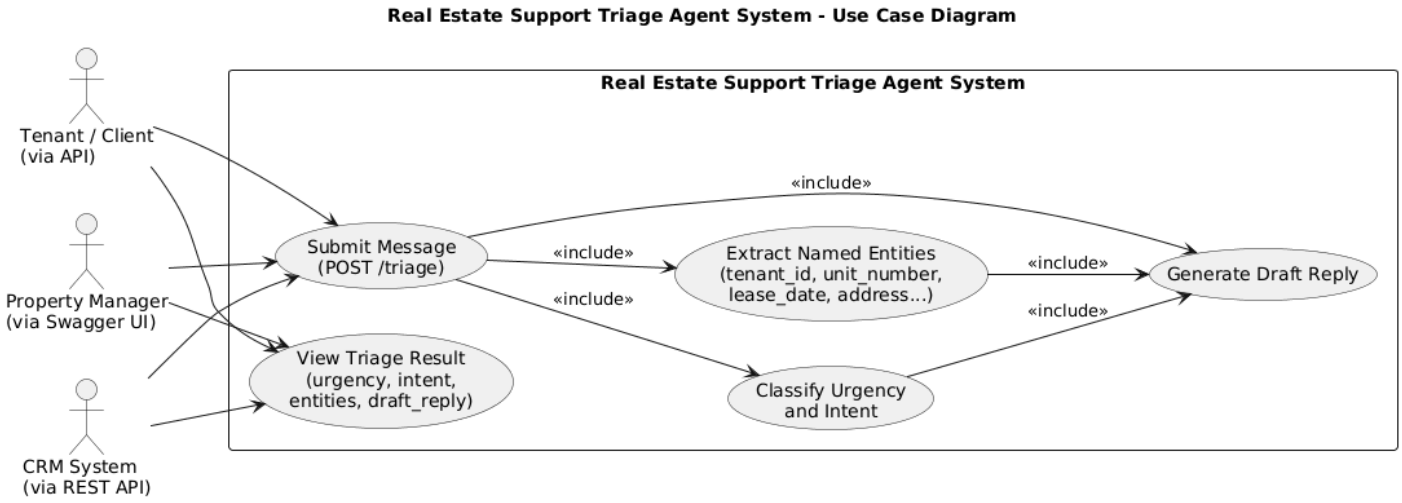
```
{
 "original_message": "Hi, I am tenant #T-4421 in Unit 3B. My heater stopped working!",
 "urgency":          "HIGH",
 "intent":           "maintenance",
 "entities": {
  "tenant_id":       "T-4421",
  "unit_number":     "3B",
  "lease_date":      null,
  "property_address": null,
  "person_name":     null,
  "dates":           null,
  "issue_type":      "heater stopped working"
 },
 "draft_reply": "Subject: Urgent Maintenance Request - Heater Issue in Unit 3B
(T-4421)\n\n...",
 "source":      "web"
}
```

3. LLD Demonstration Requirements

3.1 UML Diagrams

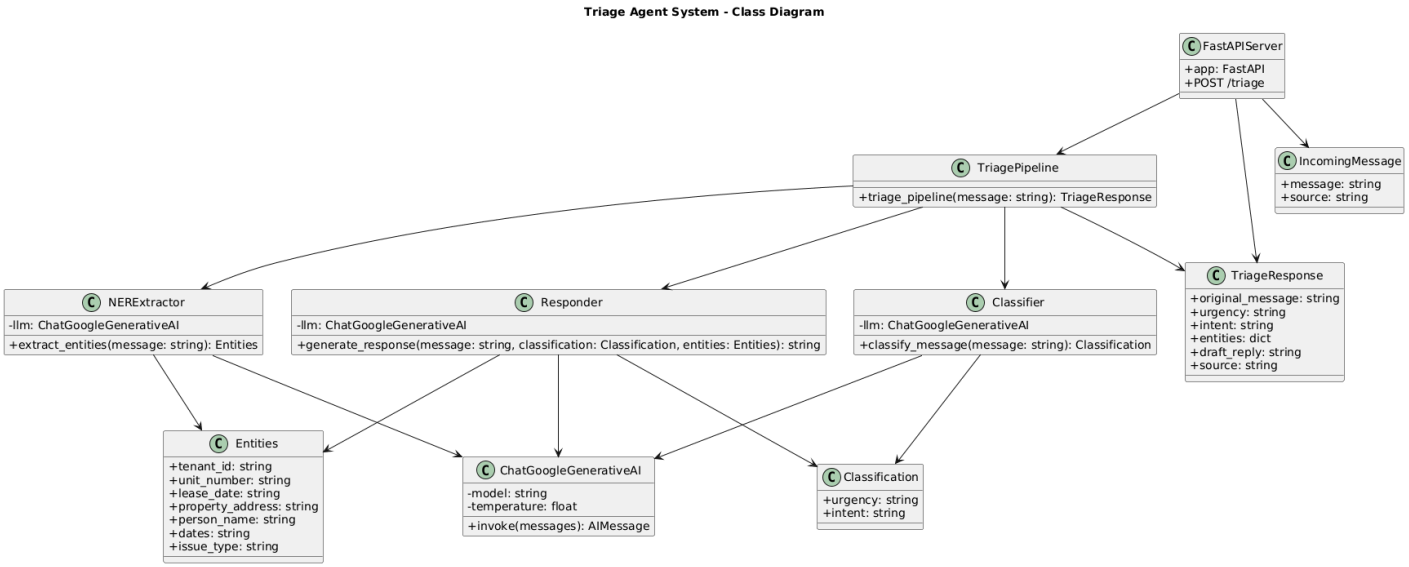
3.1.1 Use Case Diagram

The Use Case Diagram captures the interactions between external actors and the system's core functional capabilities.



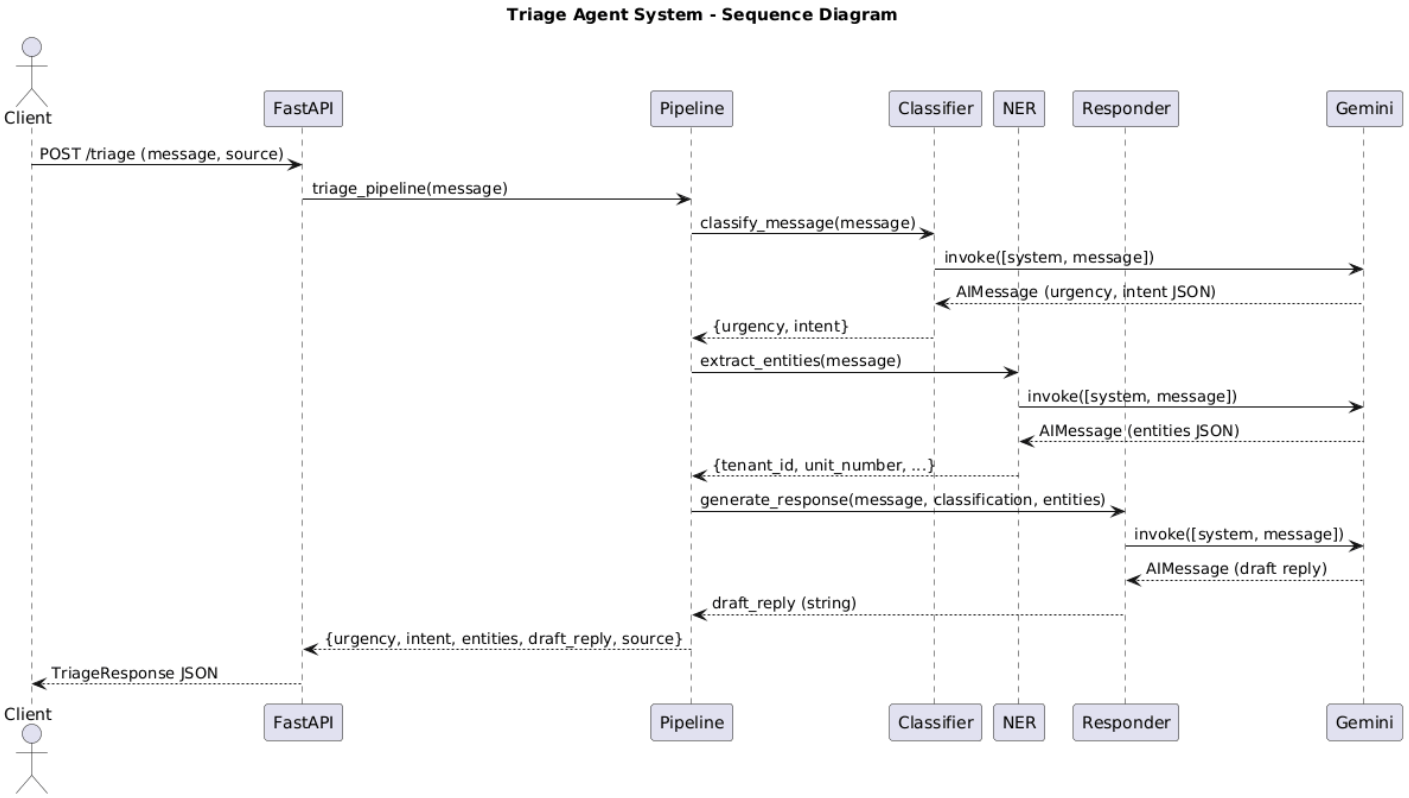
3.1.2 Class Diagram

The Class Diagram shows the key Python modules, their attributes, and the dependencies between them.



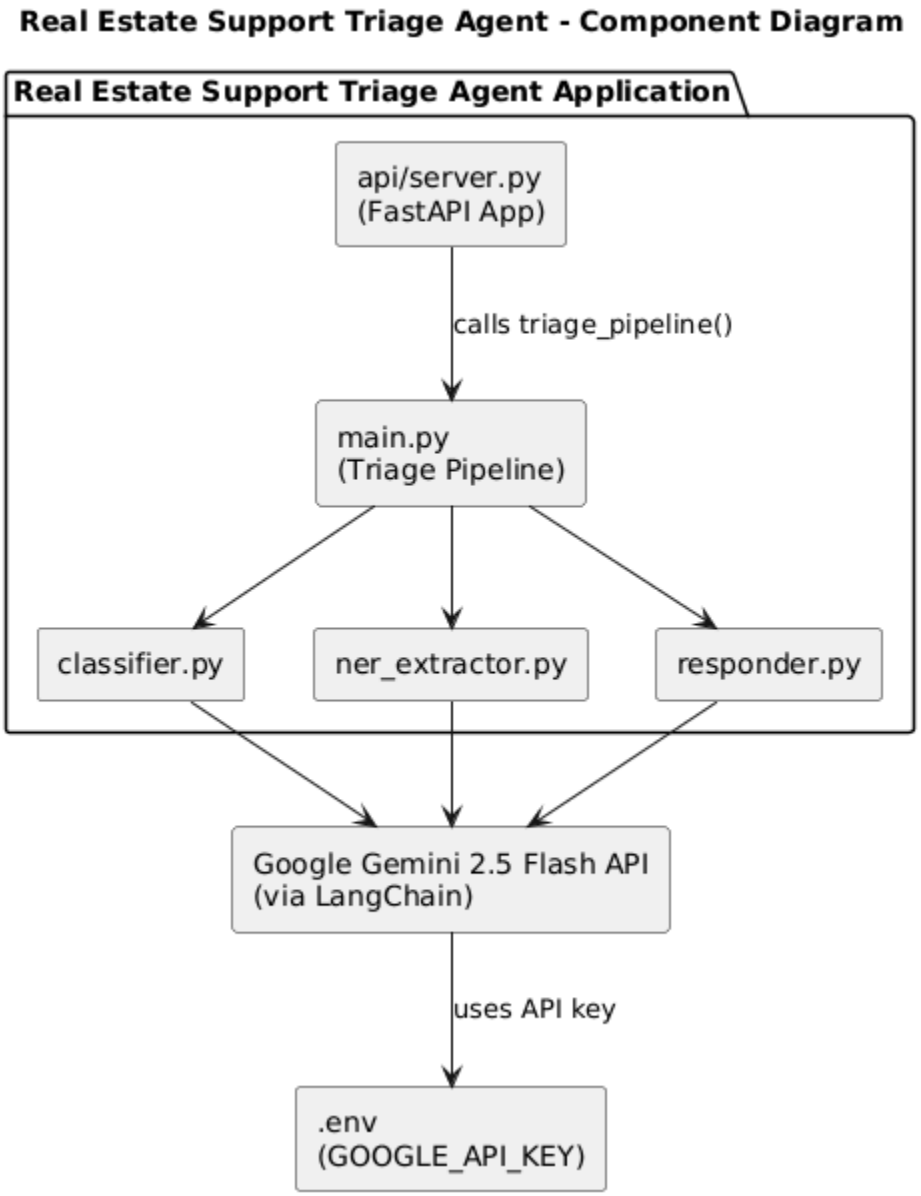
3.1.3 Sequence Diagram

The Sequence Diagram traces the full lifecycle of a single POST /triage request through all system components.



3.1.4 Component Diagram

The Component Diagram shows the physical packaging of modules and their runtime dependencies.



3.2 LLM Selection & Rationale

Google Gemini 2.5 Flash was selected as the LLM for all three pipeline stages. The decision matrix below documents the evaluation rationale against key selection criteria:

Selection Criterion	Gemini 2.5 Flash — Justification
Zero-Shot NER Capability	Gemini 2.5 Flash demonstrates strong zero-shot information extraction from unstructured text, making it capable of identifying tenant IDs, unit numbers, and dates without any fine-tuning or labelled training data.
Structured JSON Output	The model reliably produces JSON-formatted outputs when constrained by a well-formed system prompt. The clean_json() helper handles the occasional markdown fence wrapping, making the output fully machine-parseable.
Instruction Following	Gemini 2.5 Flash follows multi-rule system prompts with high fidelity, correctly applying urgency classification rules (e.g., flooding → HIGH) and adhering to the five-category intent taxonomy.
API Accessibility and Cost	Google AI Studio provides free-tier access to Gemini 2.5 Flash with 5 RPM and 1,500 RPD limits — sufficient for development, testing, and small-scale deployment without incurring infrastructure cost.
LangChain Compatibility	The langchain-google-genai package provides a first-class LangChain integration for Gemini models, enabling consistent invocation patterns (llm.invoke()), easy model swapping, and alignment with the LangChain ecosystem.
Latency	Gemini 2.5 Flash is optimised for speed rather than maximum accuracy, delivering sub-3-second response times per call. This makes the three-step pipeline complete in approximately 5–10 seconds, acceptable for draft generation workloads.
Context Window	Gemini 2.5 Flash supports a large context window, ensuring the system prompt, message content, classification output, and entity data can all be passed to the responder without truncation even for lengthy tenant messages.
Alternatives Considered	OpenAI GPT-4o and Anthropic Claude were considered. Both offer superior reasoning but require paid API access with no significant benefit for the structured extraction and classification tasks in this pipeline. Gemini 2.5 Flash provides the optimal accuracy-to-cost ratio for this use case.

3.3 Approach Toward Problem Statement

The problem statement required building a production-grade reactive triage agent for real estate communications, capable of urgency and intent classification, NER extraction, and LLM-powered draft generation. The development approach was structured into iterative phases, each building on the last:

Phase	Focus	Outcome
Phase 1	**Environment Setup**	Resolved Windows execution policy blocking venv activation. Established stable Python 3.14 environment with all required packages.
Phase 2	**Project Structure**	Created modular folder hierarchy separating agents, API, database, and tests for clear single-responsibility organisation.
Phase 3	**Gemini Connection Test**	Identified and fixed deprecated LangChain imports (langchain.schema → langchain_core.messages) and deprecated llm() call (→ llm.invoke()). Confirmed Gemini API connectivity.
Phase 4	**Agent Module Development**	Built all three agents with correct LangChain imports. Removed spaCy after DLL policy conflict. Implemented clean_json() to handle Gemini markdown fence responses.
Phase 5	**Pipeline Orchestration**	Wired all three agents into a single triage_pipeline() function with formatted CLI output and rate-limit-aware sleep guards.
Phase 6	**Full Pipeline Testing**	Validated end-to-end pipeline with three representative messages covering HIGH urgency maintenance, HIGH urgency emergency, and LOW urgency inquiry. All three produced correct classification, NER, and draft responses.
Phase 7	**Issue Resolution**	Documented and resolved six distinct issues including Windows policy errors, deprecated imports, rate limiting, DLL conflicts, and JSON parsing failures.
Phase 8	**API Server**	Wrapped pipeline behind FastAPI. Implemented Pydantic models for type-safe request/response handling. Enabled CORS and Swagger UI for interactive testing.
Phase 9+	**SQLite**	Database persistence (SQLAlchemy + SQLite), pytest unit tests, Docker containerisation, and cloud deployment (Railway / Render / AWS EC2) are planned.

4. References

S.No.	Source / Document	Description / URL
1	**Google AI Studio**	Gemini API key provisioning and free-tier quota management — https://aistudio.google.com
2	**Google Gemini 2.5 Flash Documentation**	Official model card, capabilities, and API reference — https://ai.google.dev/gemini-api/docs
3	**LangChain Documentation**	LangChain Python SDK, ChatGoogleGenerativeAI integration, message schemas — https://python.langchain.com
4	**langchain-google-genai PyPI Package**	LangChain's Google Generative AI connector — https://pypi.org/project/langchain-google-genai
5	**FastAPI Documentation**	FastAPI framework for building REST APIs with Python — https://fastapi.tiangolo.com
6	**Pydantic Documentation**	Data validation and settings management using Python type annotations — https://docs.pydantic.dev
7	**Uvicorn Documentation**	ASGI server for FastAPI applications — https://www.uvicorn.org
8	**python-dotenv**	Loads environment variables from .env file — https://pypi.org/project/python-dotenv
9	**SQLAlchemy Documentation**	Python SQL toolkit and ORM for database persistence (Phase 8, pending) — https://docs.sqlalchemy.org
10	**pytest Documentation**	Python testing framework for unit and integration tests (Phase 9, pending) — https://docs.pytest.org
11	**PowerShell Execution Policies**	Microsoft documentation on Set-ExecutionPolicy — https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/set-executionpolicy